# What Transfers Through Mathematical Distillation? A Mid-Project Report on Question Form, Dataset Construction, and Transfer

Aryama V Murthy 2024101043
Mukund Hebbar 2024101037
Yashas Nataraj Kotre 2024101028
Anivarth Varma Penumetcha 2024111006

## Abstract

A language model may be asked to solve a problem, check someone else's work, or recover a meaningful problem from incomplete mathematical evidence. These forms require related knowledge but different actions. SOLVE requires constructing a derivation. JUDGE requires checking a proposed derivation in order and identifying the first error. RECONSTRUCT requires connecting a supplied answer and partial clues to a coherent question. These abilities matter in tutoring, model review, and work with incomplete notes. We study whether distilling one form into a smaller model improves the other forms or mainly teaches the response pattern used during training. We build three disjoint 4,000-question datasets from DeepMath-103K with matched topic and difficulty distributions. DeepSeek-V4-Flash converts source questions into JUDGE and RECONSTRUCT examples. We train Qwen3.5-2B separately on each form using reverse-KL on-policy distillation from Qwen3.8-27B, with supervised warm-up for JUDGE and RECONSTRUCT. RECONSTRUCT training improves its own score by 8.7 points. JUDGE training improves identification of the first wrong step by 10.0 points. An OPD-only SOLVE run improves mean pass@1 by 4.3 points, while pass@3 does not improve. The results show that question form is an important part of what mathematical distillation teaches.

## 1 Introduction

Mathematical reasoning is often measured by whether a model produces the correct final answer. A deployed model may need to do something different with the same subject matter. A tutoring model must check a student's derivation and explain the first mistake. A model reviewing another agent's output must decide whether every step is supported. A model given incomplete notes may need to infer a well-formed question that the available equations and answer can support. A single final-answer score does not measure these abilities.

The form of a question can change what the model learns. SOLVE rewards finding and carrying out a method. JUDGE rewards checking a given method step by step. RECONSTRUCT rewards inferring missing conditions and a valid objective from partial evidence. MathBench separates several kinds of mathematical proficiency instead of treating mathematics as one score (Liu et al., 2024). ProcessBench separately tests whether a model can locate the earliest error in a solution (Zheng et al., 2025). RCoT reconstructs a problem from a generated solution and compares it with the original to expose missed conditions, unsupported assumptions, and misread objectives (Xue et al., 2023). These studies motivate evaluation beyond ordinary problem solving.

We ask how much question form matters during distillation. Distillation can train a smaller student from a larger teacher's explanations or token probabilities (Hsieh et al., 2023; Agarwal et al., 2024). However, improvement may come from learning the expected structure of one task rather than mathematical knowledge that applies to another form. Prior work also shows that training on one task can help some other tasks much more than others (Krishna et al., 2025). We therefore start a fresh copy of the same student for each task, train it on only one question form, and test it on all three forms.

This report focuses on the data needed for that comparison and reports the first complete transfer results. We use separate DeepMath questions for every task and match the topic and difficulty distributions. This reduces the chance that one task appears easier because it contains simpler questions or more familiar topics. We also report the prompt trials, conversion checks, dataset statistics, training settings, zero-training baselines, and the resulting transfer matrix.

| Task | Input | Required output |
|---|---|---|
| SOLVE | Problem | Complete solution and final answer |
| JUDGE | Problem and proposed solution | Correct or incorrect decision. If incorrect, first wrong step and explanation |
| RECON. | Supplied answer and four clues | A complete question supported by the evidence and a justification |

Table 1: The three task forms. RECON. denotes RECONSTRUCT.

| Task | Train | Test | Construction |
|---|---|---|---|
| SOLVE | 3,600 | 400 | Original question and answer |
| JUDGE | 3,600 | 400 | Problem with a checked proposed solution |
| RECON. | 3,600 | 400 | Supplied answer with four public clues |
| Total | 10,800 | 1,200 | Disjoint DeepMath source groups |

Table 2: Final experimental datasets. Topic and difficulty distributions are matched across the three rows.

## 2 Tasks and Comparison

Table 1 states what the model sees and what it must produce. The three tasks use different source questions. RECONSTRUCT follows the broad idea that a solution can reveal what problem the solver understood, but our input is a supplied answer and four partial clues rather than a complete solution.

For each training task, we begin with the same untrained student. We train one copy on that task alone and evaluate it on the fixed SOLVE, JUDGE, and RECONSTRUCT test sets. Let $S_j(M)$ be the score of model $M$ on test task $j$. If $M_0$ is the untrained student and $M_i$ is trained on task $i$, we report

$$G_{i \to j} = S_j(M_i) - S_j(M_0). \tag{1}$$

The nine score changes form a transfer matrix. Diagonal values show change on the trained task. Off-diagonal values show what happens on the other forms. We keep the two directions separate because training on SOLVE and testing on JUDGE need not match training on JUDGE and testing on SOLVE.

## 3 Dataset Construction

### 3.1 Source selection and separation

We use text-only problems from DeepMath-103K (He et al., 2026). Eligible questions have a nonempty answer, a difficulty from 5.0 to 8.0, and a broad topic of Algebra, Calculus, Discrete Mathematics, or Number Theory. Each task contains 4,000 questions, split into 3,600 training and 400 test questions. The four broad topics contribute equally within each task. Test questions are allocated across the difficulty levels in proportion to the corresponding task pool.

Each task uses a separate part of DeepMath. We group duplicate copies using source indices, normalized question text, and source-group hashes. A group can occur in only one task and one split. SOLVE excludes every source already assigned to JUDGE or RECONSTRUCT. The RECONSTRUCT test set also excludes all questions used during prompt development.

The three task datasets have matched broad-topic and difficulty distributions. The proportion at every difficulty differs by at most about one percentage point across tasks. Questions at difficulty 7.0–8.0 form 30.3–30.4% of each dataset. Fine-grained subtopics cannot be matched exactly because DeepMath has unequal supply. Their total-variation distances are 4.7% for SOLVE versus RECONSTRUCT, 11.7% for SOLVE versus JUDGE, and 12.9% for JUDGE versus RECONSTRUCT. Matching does not remove every content difference, but it controls the broad subject and difficulty differences that would otherwise directly bias the comparison.

SOLVE retains the original question and answer. The student writes a worked solution ending in one marked final answer. Deterministic checks compare that answer with the reference using normalized equality and safe substitutions. If those checks cannot establish equivalence, the result is marked unverified rather than automatically wrong.

DeepSeek-V4-Flash, served through DeepInfra, converts separate DeepMath sources into JUDGE and RECONSTRUCT records. The next sections report these pipelines in full.

## 4 JUDGE Dataset

### 4.1 Conversion procedure

For each source, we assign a correct or incorrect label. Incorrect items also receive an early, middle, or late target region for the first error. These

| Version | Main change | Usable |
|---|---|---|
| v5 | Preserve the source method | 87.5% |
| v6 | Check answer agreement and allow refusal | 75.0% |
| v7 | Re-check every step in order | 75.0% |
| v9 | Combine source checks and ordered verification | 75.0% |
| v9 ablation | Remove the few-shot examples | 90.0% |
| v10 | Require JSON and five to eight natural steps | 86.7% |
| v11 | Prefer seven or eight steps when supported | 90.0% |

Table 3: JUDGE prompt trials. Usable percentage is the primary comparison because the pilot sizes differ.

assignments are balanced within topic, difficulty, and split. The generator first checks that the source problem, answer, and reference solution agree. It can reject a source that is ambiguous, malformed, or inconsistent.

For an accepted source, the generator writes a proposed solution with five to eight numbered mathematical steps. A correct proposal must be valid throughout. For an incorrect proposal, the generator changes one natural mathematical step so that it becomes the first error. Any substantive step may be changed. All earlier steps must remain correct. Later steps may follow from the changed statement, but they cannot add an independent error. This design prevents the location or type of the mistake from being limited to a small fixed pattern.

The public example contains the problem, proposed solution, and judging instruction. The private answer contains the verdict, first wrong step, explanation, error label, and supervised target. The private information is used for training and evaluation only.

### 4.2 Prompt development

We tested prompt changes on small batches before production. Table 3 uses the percentage of attempted sources that produced mathematically usable items. A justified refusal is not counted as usable, but it is preferable to a fabricated conversion. The main improvements were a source-consistency check, permission to reject bad sources, an ordered check of all earlier steps, and a natural five-to-eight-step requirement. A matched ten-source ablation without examples had the same accept or refuse decisions as the full prompt on those sources. We kept the examples because they make the output format and error annotation explicit.

The production model was `DeepSeek-V4-Flash-0731` with temperature 0.1, low reasoning effort, JSON response mode, and an initial limit of 8,192 output tokens. Full regeneration used 16,384 tokens. Requests ran with concurrency 150. JSON repair was limited to syntax and field formatting. It never changed the mathematics. A mathematical failure caused regeneration of the whole item.

An independent Qwen3-235B-A22B-Instruct-2507 verifier checked the verdict, every step before the stated error, the first wrong step, and agreement with the source answer. Items that failed these checks were retried or excluded. The first 4,000 assignments used v10. The final 1,000 used v11 to increase the number of naturally supported seven-step and eight-step solutions.

| Topic | Total | Correct | Incorrect |
|---|---|---|---|
| Algebra | 1,143 | 558 | 585 |
| Calculus | 1,194 | 592 | 602 |
| Discrete Mathematics | 1,117 | 538 | 579 |
| Number Theory | 1,150 | 570 | 580 |
| All | 4,604 | 2,258 | 2,346 |
| Share | 100% | 49.04% | 50.96% |

Table 4: Topic and label counts in the verified JUDGE release pool. Topic shares are 24.83%, 25.93%, 24.26%, and 24.98% in row order.

### 4.3 Final statistics

The conversion retained 4,604 of 5,000 assignments, a 92.08% yield. This verified release pool contains 3,619 training candidates and 985 test candidates. It has 4,604 unique source IDs. A matched replacement source was needed for 95 items, or 2.06%. From this pool, we select the balanced experimental subset of 3,600 training and 400 test examples used in the transfer study. That subset has 1,800 correct and 1,800 incorrect training examples and 200 of each label at test time.

The complete 4,604-item pool has mean difficulty 6.23 and mean proposed-solution length 6.26 steps. It covers 42 detailed topic paths. Broad topics are close to equal and the labels are close to balanced, as shown in Table 4.

Incorrect examples are divided almost equally by error region. There are 781 early errors, 789 middle errors, and 776 late errors. These are 33.29%, 33.63%, and 33.08% of the incorrect items. The exact first-error distribution from steps 1 through 8 is 6.14%, 23.87%, 16.45%, 15.73%, 19.74%, 14.71%, 3.15%, and 0.21%. Table 6 shows that the

| Steps | Total | Data share | Correct | Incorrect |
|---|---|---|---|---|
| 5 | 343 | 7.45% | 73 | 270 |
| 6 | 2,795 | 60.71% | 1,491 | 1,304 |
| 7 | 1,372 | 29.80% | 651 | 721 |
| 8 | 94 | 2.04% | 43 | 51 |

Table 5: JUDGE label counts by solution length. The incorrect rates within the four rows are 78.72%, 46.65%, 52.55%, and 54.26%.

| Difficulty | Count | Dataset share | Incorrect share |
|---|---|---|---|
| 5.0 | 917 | 19.92% | 50.38% |
| 5.5 | 839 | 18.22% | 50.06% |
| 6.0 | 995 | 21.61% | 50.95% |
| 6.5 | 438 | 9.51% | 51.14% |
| 7.0 | 479 | 10.40% | 52.82% |
| 7.5 | 327 | 7.10% | 49.85% |
| 8.0 | 609 | 13.23% | 52.05% |

Table 6: Difficulty and label distribution in the JUDGE release pool.

incorrect share also stays close to one half at every difficulty.

The main known imbalance is solution length. Five-step proposals are incorrect much more often than the other lengths. We therefore report JUDGE performance by length and will compare it with a step-count-only baseline. This correlation must not be mistaken for mathematical checking ability.

# 5 RECONSTRUCT Dataset

## 5.1 Task and source allocation

A RECONSTRUCT example hides a DeepMath question and shows the student its final answer and four clues. The student must write a complete question that the clues support and whose answer is the supplied answer. Every clue must be supported by the source. Together, the clues must not copy or closely paraphrase the question, hide a missing value inside an arithmetic expression, or give the full solution. At least one premise, parameter, relation, or requested quantity must stay hidden. Using the answer to recover a missing premise is allowed.

We sampled 38,000 DeepMath rows in 380 random blocks of 100. We first excluded 1,519 questions used in earlier RECONSTRUCT experiments. Filtering then removed 7,205 rows outside the four topics or the 5.0–8.0 range, 465 duplicate or already-used questions, 18 unresolvable multiple-choice answers, and one row without a reference solution. This left 30,311 candidates. Supply was uneven, with 12,181 Calculus candidates but only

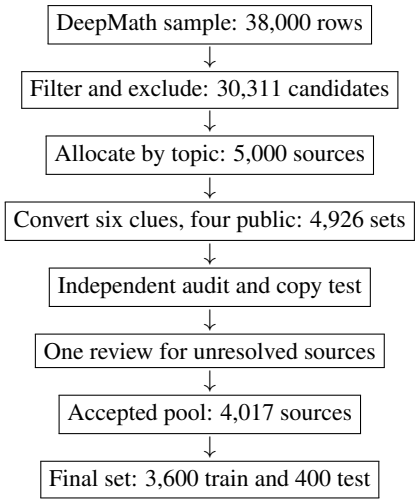



Figure 1: RECONSTRUCT generation pipeline. The 983 dropped sources are 74 failed conversions, 486 sources judged unrepairable, and 423 failed review calls.

1,607 for Discrete Mathematics. We drew 1,250 sources per topic, split into 1,000 training and 250 test sources. We did not balance difficulty within topics because equal cells would have reduced every cell to the smallest one, the upper difficulty band of Number Theory.

## 5.2 Generation pipeline

Figure 1 shows the three model steps. Every call uses DeepSeek-V4.1-Flash with temperature 0 and low reasoning effort. No student output is used during generation.

**Step 1: convert.** The converter sees the source question, answer, and DeepMath's first reference solution, which we treat as unverified. It either rejects the source or returns six ordered clues, each with a private note identifying its support in the source. The first four clues are public and the last two are spares for Step 3. Two worked examples show the two ways to withhold information. One hides a premise that the student recovers from the answer. The other hides which quantity is requested.

**Step 2: two independent checks.** A source-aware audit sees the source, reference solution, answer, and clues. It checks every clue, looks for a copied question, a disguised value, or a full solution, and must name the hidden element. A source-blind copy test sees only the answer and clues. It tries to rebuild a question by copying, paraphrasing, renaming, and reordering, without adding a new condition or target. A clue set passes only if the

| Step 2 outcome | Sources |
|---|---|
| Accepted by both checks | 1,991 |
| Rejected by both checks | 1,116 |
| Rejected by copy test only | 496 |
| Rejected by audit only | 433 |
| One rejection, other call failed | 393 |
| Unresolved because a call failed | 496 |
| Audit requested review | 1 |
| All converted sources | 4,926 |

Table 7: RECONSTRUCT Step 2 outcomes.

audit accepts it and the copy test cannot rebuild a question. Table 7 shows that the copy test caught 496 clue sets that the source-aware audit accepted.

**Step 3: review and correct.** Step 2 accepted 1,991 sources. Each of the other 2,935 received one more call. The reviewer sees the source, the four public clues, the two spares, and both Step 2 reviews. It can accept the clues unchanged, correct them, or mark the source unrepairable. A correction keeps every sound clue word for word. It removes leaked content by withholding it, not by rewording it. A forty-source trial rescued nine rejected sources, after which the same prompt was used unchanged. To limit cost, this decision is final.

One caught leak came from a recurrence with $a_1 = 1$ and $a_{n+1} = (1 + a_1) \cdots (1 + a_n)$. The clue "the recurrence implies $a_{n+1} = a_n(1 + a_n)$" holds at $n = 1$ only when $a_1 = 1$, so it revealed the hidden starting value. The corrected clue adds "for $n \geq 2$."

### 5.3 Development trials

We selected this design through small experiments on development sources that never enter the test set. A local 4-bit Qwen3-4B student was graded by DeepSeek. A grounded success is a well-posed question with the supplied answer that uses every clue. A recovery also matches the original question. Fewer clues let more sources pass without producing more recoveries, so we kept four. Removing the copy test admitted near-copies, so we kept both checks. Changing prompt wording or the student's seed gave no reliable gain.

### 5.4 Final statistics

The pipeline accepted 4,017 sources, and we kept 4,000 to match the topic and difficulty profile of the other tasks. All 400 test items come from base test sources that no development step used. The training split uses 3,197 base training sources and 403 base test sources that the test split did not need. Table 9 gives the acceptance route of the kept items.

| Trial | Main change | Result |
|---|---|---|
| First pilot | One to four clues | 3/8 and 1/12 grounded successes |
| Three clues | Show first three clues | 7/16 passed, no recoveries |
| Looser audit | Relax leakage rule | 6/16 passed, no recoveries |
| Four clues | First four of six | 52/100 passed, mean 44.4 |
| No copy test | Remove blind check | At least 2/8 near-copies |
| Correction | One review call | 9/40 rescued |

Table 8: RECONSTRUCT development trials.

| Group | Step 2 | Fixed | Kept | Total |
|---|---|---|---|---|
| Train | 1,780 | 1,152 | 668 | 3,600 |
| Test | 201 | 119 | 80 | 400 |
| Algebra | 503 | 324 | 193 | 1,020 |
| Calculus | 540 | 324 | 160 | 1,024 |
| Discrete Math. | 404 | 320 | 238 | 962 |
| Number Theory | 534 | 303 | 157 | 994 |
| Lower band | 1,553 | 784 | 448 | 2,785 |
| Upper band | 428 | 487 | 300 | 1,215 |
| All | 1,981 | 1,271 | 748 | 4,000 |
| Share | 49.5% | 31.8% | 18.7% | 100% |

Table 9: Acceptance route of the 4,000 retained RECONSTRUCT items. The lower band is difficulty 5.0–6.5 and the upper band is 7.0–8.0.

Harder sources depend more on Step 3. It supplied 64.8% of upper-band items and 44.2% of lower-band items. The 1,271 corrected items changed 1,753 clues. One clue changed in 877 items and two changed in 316. The recorded reasons were question disclosure in 1,296 changed clues, solution disclosure in 223, padding in 141, unsupported content in 76, and a disguised value in 11. Corrected items were not checked again before inclusion. A later audit and copy test passed 508, flagged 549, and has no result for 214. All 748 items accepted unchanged had been rejected or left unresolved by Step 2.

A public clue has 15.4 words on average, and a set of four has 61.6 words. The hidden questions have 33.1 words on average. The items cover 49 detailed topic paths with mean difficulty 6.21, and 47.3% of the supplied answers are integers.

### 5.5 Records and scoring

The public record contains only the answer, four clues, and instruction. The private record contains

| Setting | Value |
|---|---|
| Student and teacher | Qwen3.5-2B, Qwen3.8-27B |
| Adapter | LoRA rank 64, $\alpha = 64$, dropout 0 |
| SFT data | 470 responses for JUDGE and RECON. |
| SFT optimization | 1 epoch, batch 16, 30 steps |
| SFT learning rate | $10^{-4}$ with cosine decay |
| OPD data | 16 steps $\times$ 32 new prompts |
| OPD learning rate | Constant $5 \times 10^{-5}$ |
| Rollout sampling | Temperature 1, top-$p$ 1 |

Table 10: Training configuration. SOLVE uses only the OPD rows.

the source question, acceptance route, Step 3 findings, and original clues for corrected items. A source-aware grader checks whether the output preserves the source premises and asks for the same quantity. A source-blind grader checks whether the new question is well formed, uses the clues, and is mathematically valid.

## 6 Models and Training

The student is Qwen3.5-2B and the teacher is Qwen3.8-27B. Both use text-only, non-thinking mode with bfloat16 weights. We checked all 248,070 shared token IDs and found identical token strings. Rendered prompts also matched token for token. This matters because the student and teacher probabilities must refer to the same next-token choices.

Every task run starts from a fresh student. LoRA adapters are applied to all attention and MLP projections with rank 64, scaling parameter $\alpha = 64$, and dropout 0. JUDGE and RECONSTRUCT use supervised fine-tuning followed by reverse-KL on-policy distillation (Agarwal et al., 2024). The revised SOLVE result uses OPD only, without the supervised warm-up.

For supervised fine-tuning on JUDGE and RECONSTRUCT, 512 questions per task are reserved for teacher-response generation and do not overlap evaluation or on-policy prompts. The teacher produces two samples per question at temperature 1.0, top-$p$ 1.0, and a 16,384-token limit. We keep the first completed sample that follows the required format and use the first 470 retained responses. The student trains for one epoch on response tokens only. Prompt tokens are masked. Batch size 16 gives 30 optimizer steps. AdamW uses learning rate $10^{-4}$, three warm-up steps, cosine decay to $10^{-5}$, weight decay 0, and gradient clipping at 1.0.

| Metric | Student | Teacher |
|---|---|---|
| SOLVE mean pass@1 | 52.5% | 79.5% |
| JUDGE verdict | 57.5% | 73.5% |
| JUDGE first wrong step | 18.0% | 55.5% |
| JUDGE rubric | 47.0 | 78.1 |
| RECONSTRUCT rubric | 33.5 | 70.1 |

Table 11: Zero-training results. SOLVE is mean pass@1 over three samples for the student. Rubric scores are out of 100. The first-wrong-step metric uses the 200 incorrect JUDGE items.

On-policy distillation runs for 16 steps. JUDGE and RECONSTRUCT start from their supervised adapters. SOLVE starts from the untrained student. Each step uses 32 distinct prompts and one current-student rollout per prompt, for 512 questions used once. Rollouts use temperature 1.0, top-$p$ 1.0, and a 16,384-token limit. At each generated token, the loss moves the student distribution toward the teacher distribution on the student's own response history. Importance weights are capped at 2.0. AdamW uses a constant learning rate of $5 \times 10^{-5}$, weight decay 0, and gradient clipping at 1.0. The adapter is synchronized with the generation server after every update. Training curves for these runs are linked in Appendix B.

## 7 Evaluation and Baselines

Each task has a fixed 400-question test set. JUDGE contains 200 correct and 200 incorrect proposed solutions. Generation uses temperature 1.0, top-$p$ 1.0, top-$k$ 0, one sample, seed 0, no presence or repetition penalty, and at most 32,768 new tokens.

SOLVE uses deterministic final-answer checks. JUDGE first parses the decision and error step, then uses a DeepSeek-V4.1-Flash rubric for the verdict, location, explanation, and mathematical validity. RECONSTRUCT uses a four-part rubric for source fidelity, task consistency, clue support, and mathematical validity. Each part is scored from 0 to 4, and the total is 6.25 times their sum. An output that only copies the answer or contains no question scores 0. Open-ended grading uses temperature 0 and low reasoning effort.

Table 11 gives the zero-training baselines. The gap is largest on locating an error and reconstructing a supported question.

## 8 Transfer Results

Table 12 reports the final task-specific checkpoints. SOLVE uses the OPD-only rerun. JUDGE and RE-

CONSTRUCT use supervised fine-tuning followed by OPD. Each parenthesized value is the paired change from the corresponding untrained student when that paired comparison is available.

RECONSTRUCT training produces the clearest same-task improvement. Its 8.7-point gain has an interval entirely above zero. JUDGE training improves the complete rubric by 4.0 points, but its interval includes a small negative value. The more direct first-wrong-step accuracy rises from 18.0% to 28.0%, a 10.0-point gain with a 95% interval of $[+5.0, +15.0]$. Verdict accuracy changes from 57.5% to 56.5%, so the improvement comes from locating errors rather than making more binary decisions.

The OPD-only SOLVE model improves mean pass@1 from 52.5% to 56.8%. The paired gain is 4.3 points with a 95% interval of $[+1.3, +7.5]$. Pass@2 changes from 68.0% to 68.9%, a 1.0-point change with interval $[-2.2, +4.1]$. Pass@3 changes from 74.9% to 74.2%, a $-0.6$-point change with interval $[-4.2, +2.9]$. The model therefore improves average single-sample accuracy but does not solve a broader set of questions across three attempts. It solves 297 questions at least once, compared with 295 for the untrained model. JUDGE and RECONSTRUCT training each score 43.5% on the earlier one-sample SOLVE evaluation. Most other cross-task changes have intervals that include zero.

The transfer is also asymmetric. RECONSTRUCT training improves RECONSTRUCT but lowers SOLVE. JUDGE training improves first-error identification but does not improve RECONSTRUCT. The OPD-only SOLVE checkpoint still requires cross-task evaluation. The available results support the main hypothesis that training form affects what the student learns, even when mathematical topics and difficulty are controlled.

## 9 Conclusion

We compare three ways of asking about the same mathematical subjects. The datasets use separate DeepMath questions and matched topic and difficulty distributions. The completed transfer results show useful improvement within RECONSTRUCT and in JUDGE error location, but little evidence that these improvements carry to the other forms. Training on JUDGE or RECONSTRUCT also lowers ordinary SOLVE accuracy. These results make question form a necessary experimental variable when distilling mathematical reasoning.

## 10 Remaining Timeline

Table 13 gives the remaining experimental sequence. Later stages use the results of earlier stages. Controlled comparisons keep the student, evaluation sets, and training budget fixed.

| Order | Work and comparison |
|---|---|
| 1 | Finish distillation training on the full SOLVE, JUDGE, and RECONSTRUCT datasets and complete the missing cross-task evaluations. |
| 2 | Test training with GRPO and compare it with the current OPD results under the same data and update budget. |
| 3 | Compare single-task training with an equal mixed-task run and a mixture chosen from the transfer results. |
| 4 | Test curriculum effects by changing which task type is trained first. Compare each ordered curriculum with a shuffled mixture containing the same examples. |
| 5 | Develop dataset augmentation methods from the observed errors and transfer results. Test the selected methods with the existing validity and balance checks. |
| 6 | Identify the training strategy that gives the strongest overall cross-task transfer and generalization using evidence from the preceding experiments. |

Table 13: Timeline for the remaining experiments.

## Limitations

The JUDGE and RECONSTRUCT data are generated and checked by models. Independent checks reduce errors but do not replace a human audit. RECONSTRUCT has 549 corrected examples flagged by later checks and 214 without recorded follow-up checks. Teacher responses for supervised fine-tuning were selected for completion and format, not independently verified mathematical correctness.

The JUDGE release pool is nearly balanced by topic, difficulty, label, and error region, but step length still correlates with the label. The final production batch also uses v11 while earlier batches use v10. RECONSTRUCT and part of JUDGE evaluation depend on model grading, so these scores need calibration against human judgments.

The transfer study currently uses one training run per task. Output truncation is also a concern for the JUDGE-trained checkpoint. Its cap rates are 13.8% on SOLVE, 3.0% on JUDGE, and 18.0% on RECONSTRUCT. For the RECONSTRUCT-trained checkpoint they are 4.0%, 0.2%, and 6.8%.

| Training data | SOLVE accuracy (%) | JUDGE rubric (/100) | RECONSTRUCT (/100) |
|---|---|---|---|
| None | 52.5 | 47.0 | 33.5 |
| SOLVE, OPD only | 56.8 ($+4.3$ [$+1.3$, $+7.5$]) | – | – |
| JUDGE | 43.5 ($-9.0$) | 51.0 ($+4.0$ [$-0.1$, $+7.9$]) | 32.5 ($-0.6$ [$-4.3$, $+3.2$]) |
| RECONSTRUCT | 43.5 ($-9.0$) | 45.2 ($-1.9$ [$-5.3$, $+1.5$]) | 42.4 ($+8.7$ [$+5.5$, $+12.3$]) |

Table 12: Transfer after training on one task form. SOLVE uses OPD only. Its cross-task evaluations have not yet been rerun. JUDGE and RECONSTRUCT use supervised fine-tuning followed by OPD. The $-9.0$ SOLVE changes are arithmetic differences. Their paired intervals were not recomputed under the three-sample protocol.

Some lower cross-task scores may therefore reflect unstable generation length as well as weaker reasoning.

## References


Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. 2026. DeepMath-103K: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. In *International Conference on Learning Representations*.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017. Association for Computational Linguistics.

Shambhavi Krishna, Atharva Naik, Chaitali Agarwal, Sudharshan Govindan, Haw-Shiuan Chang, and Taesung Lee. 2025. Latent traits and cross-task transfer: Deconstructing dataset interactions in LLM fine-tuning. In *Proceedings of the 14th Joint Conference on Lexical and Computational Semantics*, pages 225–241. Association for Computational Linguistics.

Hongwei Liu, Zilong Zheng, Yuxuan Qiao, Haodong Duan, Zhiwei Fei, Fengzhe Zhou, Wenwei Zhang, Songyang Zhang, Dahua Lin, and Kai Chen. 2024. MathBench: Evaluating the theory and application proficiency of LLMs with a hierarchical mathematics benchmark. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 6884–6915. Association for Computational Linguistics.

Tianci Xue, Ziqi Wang, Zhenhailong Wang, Chi Han, Pengfei Yu, and Heng Ji. 2023. RCoT: Detecting and rectifying factual inconsistency in reasoning by reversing chain-of-thought. *arXiv preprint arXiv:2305.11499*.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 1009–1024. Association for Computational Linguistics.

## A Reproducibility and Prompt Record

Code, dataset scripts, and the full prompts are available at `MukundHebbar/anlp-project`. The summary below records the instructions needed to interpret the reported data and scores.

**SOLVE student instruction.** Solve the problem, show the derivation, and put only the answer in one `<final_answer>` tag.

**JUDGE instructions.** The student checks numbered steps and returns a decision, first wrong step, and explanation. The generator first rejects unsuitable sources. It otherwise writes five to eight steps in JSON. Incorrect items contain one natural first error and preserve the correct prefix. Private fields store the verdict, error step, explanation, and error type.

**RECONSTRUCT instructions.** The student uses an answer and four clues to write a supported mathematics question and justify the answer. The generator rejects unsuitable sources or creates six clues. Four are public and two are reserved for correction. Clues must support a valid reconstruction without copying the source question or solution.

**Grading prompts.** JUDGE scores the verdict, error location, explanation, and validity. RECONSTRUCT separately checks source fidelity and whether the question is well formed, supported, and valid.

## B Training Logs

Graphs from the training runs described in Section 6, with the configuration in Table 10, are available in a shared Google Drive folder (`training_logs_anlp`):

`https://drive.google.com/drive/folders/102U2-rUv_gCTFx6efWT1BcIhW7Nh2bCA`

These graphs are supplementary. All results reported in this paper are stated in the main text.